Encontro 3: Fundamentos do Framework Vue.js

Sumário

Encontro 3: Fundamentos do Framework Vue.js 1

1. Introdução 2
2. A Arquitetura Baseada em Componentes 2
   2.1. Vantagens da Componentização: 2
3. Anatomia de um Componente Vue (Single File Component - SFC) 3
   3.1. O Bloco <template> 3
   3.2. O Bloco <script> 4
   3.3. O Bloco <style> 5
4. Exercício Prático: Criando um Componente de Perfil 6

1. Introdução

Nos encontros, conhecemos as alternativas de uso do Vue.js e instalamos o ambiente de desenvolvimento, além de testar nosso primeiro projeto Vue.js a partir de um repositório GitHub. Agora, vamos aprofundar nosso conhecimento nos blocos de construção fundamentais de qualquer aplicação Vue: a arquitetura de componentes e a estrutura dos arquivos `.vue`.

2. A Arquitetura Baseada em Componentes

O Vue.js, assim como outros frameworks modernos, utiliza uma arquitetura baseada em componentes. Pense em componentes como peças de Lego. Cada peça tem sua própria estrutura, lógica e aparência, e podemos combiná-las para construir interfaces complexas. Um site pode ser dividido em componentes como Header, Sidebar, ProductList, Footer, etc.

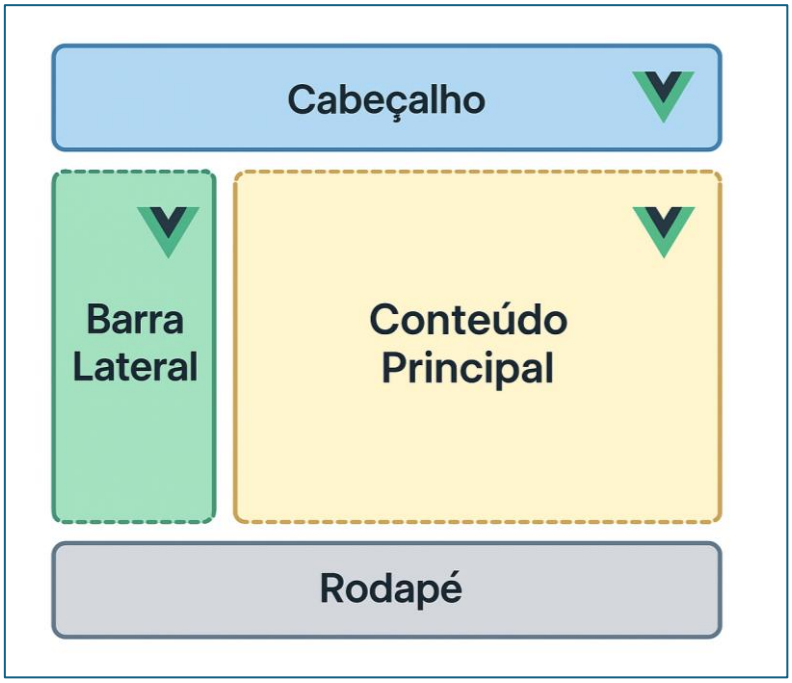


Figura 1: Visualização de uma página web dividida em componentes reutilizáveis como Header, Sidebar, Main Content e Footer.

2.1. Vantagens da Componentização:

- Reutilização: Um componente criado uma vez, como um botão customizado, pode ser reutilizado em várias partes da aplicação.
- Manutenção: Isolar a lógica em componentes torna mais fácil encontrar e corrigir bugs, pois o escopo do problema é menor.
- Escalabilidade: É mais simples adicionar novas funcionalidades à aplicação construindo novos componentes ou compondo os existentes.
- Legibilidade: O código se torna mais organizado e fácil de entender, pois cada arquivo tem uma responsabilidade única.

3. Anatomia de um Componente Vue (Single File Component - SFC)

O Vue popularizou o conceito de Single File Components (SFCs), ou Componentes de Arquivo Único. A ideia é encapsular o HTML, o JavaScript e o CSS de um componente em um único arquivo com a extensão `.vue`. Isso mantém tudo o que está relacionado a um componente no mesmo lugar.

Um arquivo `.vue` é tipicamente dividido em três blocos principais:

- `<template>`: Onde fica a estrutura HTML do componente.
- `<script>`: Onde fica a lógica JavaScript do componente.
- `<style>`: Onde fica a estilização CSS do componente.



Figura 2: Diagrama da estrutura de um arquivo `.vue` (Single File Component)

A figura 2 mostra os três blocos principais: `<template>` para o HTML, `<script>` para a lógica JavaScript e `<style>` para o CSS.

3.1. O bloco <template>

Este bloco contém o HTML que define a estrutura visual do seu componente. O Vue utiliza uma sintaxe de template que é uma extensão do HTML, permitindo-nos conectar a estrutura do DOM aos dados da nossa aplicação de forma reativa.

- Interpolação de Texto: Para exibir dados dinâmicos, usamos a sintaxe de chaves duplas, também conhecida como "Mustache syntax".

```
<template>
  <h1>{{ titulo }}</h1>
</template>
```

Neste exemplo, o conteúdo da tag `<h1>` será o valor da propriedade `titulo` definida no bloco `<script>`.

3.2. O bloco <script>

É aqui que a mágica acontece. O bloco `<script>` contém toda a lógica do nosso componente. Exportamos um objeto JavaScript que define o comportamento do componente.

- `export default {}`: Todo SFC exporta um objeto de configuração.
- `name`: É uma boa prática nomear seus componentes. Isso é útil para depuração.
- `data()`: Esta função é fundamental. Ela retorna um objeto contendo os dados reativos do componente. Sempre que um valor dentro de `data` muda, o Vue automaticamente atualiza o template para refletir essa mudança.

```
<script>
export default {
  name: 'MeuComponente',
  data() {
    return {
      titulo: 'Minha Primeira Aplicação Vue',
      contador: 0
    }
  }
}
</script>
```

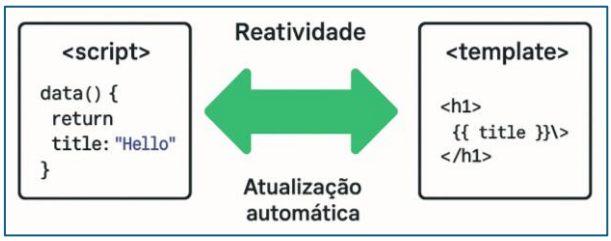


Figura 3: Ilustração do conceito de Data Binding.

Na figura 3, uma seta bidirecional conecta o objeto `data` no bloco `<script>` com o `<template>`, mostrando que mudanças em um refletem no outro.

- `methods`: Este objeto contém as funções que podem ser chamadas a partir do template (por exemplo, em resposta a um clique de botão). Métodos têm acesso aos dados do componente através de `this`.

```
<script>
export default {
  // ... name e data ...
  methods: {
    incrementar() {
      this.contador++;
    }
  }
}
</script>
```

3.3. O bloco <style>

Este bloco é usado para estilizar o componente. É apenas CSS comum, mas com um superpoder: o atributo `scoped`.

- CSS Escopado (`scoped`)**:** Quando você adiciona o atributo `scoped` à tag `<style>`, o Vue limita as regras de CSS apenas a este componente. Isso é feito adicionando um atributo de dados único aos elementos do componente e modificando os seletores CSS para mirar nesses atributos. Isso evita que os estilos de um componente "vazem" e afetem outros componentes.

```
<style scoped>
h1 {
  color: #42b983; /* Estilo só se aplicará aos h1s deste componente */
}

button {
  background-color: #35495e;
  color: white;
  padding: 10px;
  border: none;
  border-radius: 5px;
}
</style>
```



Figura 4: Diagrama mostrando dois componentes lado a lado.

Na figura 4, o componente da esquerda tem um estilo `h1 { color: blue; }` escopado, e seu H1 é azul. O componente da direita tem um estilo `h1 { color: red; }` escopado, e seu H1 é vermelho, demonstrando que os estilos não interferem um no outro.

4. Exercício Prático: Criando um Componente de Perfil

Vamos aplicar este conteúdo criando um componente simples que exibe o perfil de um usuário.

a) Crie um novo componente: na pasta `src/components`, crie um arquivo chamado `PerfilUsuario.vue`.

b) Defina o template: no bloco `<template>`, adicione a estrutura para exibir o nome, a idade e uma imagem do usuário.

```
<template>
  <div class="perfil">
    <img :src="imagemUrl" alt="Foto do Perfil">
    <h2>{{ nome }}</h2>
    <p>Idade: {{ idade }}</p>
    <button @click="aumentarIdade">Fazer Aniversário</button>
  </div>
</template>
```

c) Adicione a lógica: no bloco `<script>`, defina os dados do usuário e o método para incrementar a idade.

```
<script>
export default {
  name: 'PerfilUsuario',
  data() {
    return {
      nome: 'Maria Silva',
      idade: 29,
      // Serviço de imagens de avatar
      imagemUrl: 'https://i.pravatar.cc/150?img=22'
    }
  },
  methods: {
    aumentarIdade() {
      this.idade++;
    }
  }
}
</script>
```

d) Estilize o componente: Adicione alguns estilos escopados no bloco `<style scoped>`.

```
<style scoped>
.perfil {
  border: 1px solid #ccc;
  border-radius: 8px;
  padding: 20px;
  text-align: center;
  width: 200px;
  margin: 20px;
```

```
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
img {
  width: 100px;
  height: 100px;
  border-radius: 50%;
}
</style>
```

e) Use o componente: finalmente, importe e use o seu novo componente `PerfilUsuario.vue` dentro do `App.vue`.

```
// Em App.vue
<template>
  <div id="app">
    <h1>Nossa Equipe</h1>
    <PerfilUsuario />
  </div>
</template>

<script>
import PerfilUsuario from './components/PerfilUsuario.vue';

export default {
  name: 'App',
  components: {
    PerfilUsuario
  }
}
</script>
```

Ao rodar a aplicação, você verá o componente de perfil renderizado. Ao clicar no botão, a idade do usuário será atualizada na tela automaticamente, demonstrando o poder da reatividade do Vue.js.